

Laporan Praktikum 1 Data Structures and Algorithm Analysis Kelas LAB

Nama : Mohamad Faizal Norhavid
NIM : 2902761311

Pada Praktikum ini, Mahasiswa diberikan tugas untuk membuat operasi sederhana array menggunakan tipe data array primitive dan array object bawaan java ArrayList dan membandingkan mana yang lebih cepat dan optimal pada banyak data

A. Studi Literatur

1) Tipe data primitive

Berdasarkan dokumentasi resmi Java, tipe data primitive adalah tipe data paling dasar yang didukung langsung oleh bahasa pemrograman Java. Tipe data ini tidak bertindak sebagai objek, melainkan secara langsung menyimpan nilai datanya di dalam memori. Beberapa contoh tipe data primitif yang sering digunakan meliputi int, double, boolean, dan char. Dalam praktikum ini, penggunaan array dengan tipe data primitif (seperti int[]) menawarkan keunggulan utama pada efisiensi memori dan kecepatan eksekusi. Karena ukurannya bersifat statis sejak awal dideklarasikan dan datanya disimpan secara berurutan (contiguous) di memori, operasi seperti penelusuran (traversal), pencarian, maupun alokasi memori tingkat rendah dapat dieksekusi dengan sangat cepat oleh Java Virtual Machine

2) Tipe data built-in

Selain tipe data primitif, Java juga menyediakan struktur data bawaan atau built-in berupa class objek yang dikelompokkan dalam Java Collections Framework pada package java.util. Merujuk pada literatur dari GeeksforGeeks, salah satu implementasi yang paling umum digunakan adalah ArrayList. Berbeda dengan array primitif yang ukurannya mutlak, ArrayList adalah array yang bersifat dinamis. Ukurannya dapat merenggang atau menyusut secara otomatis ketika sistem menambahkan atau menghapus sebuah elemen. Penggunaan struktur built-in ini sangat memudahkan programmer karena telah dilengkapi dengan berbagai fungsi (metode) bawaan yang siap pakai, seperti proses searching dan deletion tanpa perlu menggeser indeks secara manual. Walaupun menawarkan tingkat fleksibilitas yang tinggi dan penulisan kode yang lebih ringkas, ArrayList umumnya memiliki sedikit beban tambahan (overhead) pada performa waktu dibandingkan array primitif, karena ArrayList hanya dapat membungkus data dalam bentuk objek (misalnya Integer), bukan primitif murni.

3) Java Runnable

Runnable adalah sebuah interface fungsional inti di Java yang dirancang agar sebuah instansi kelas dapat dieksekusi oleh sebuah thread. Interface ini sangat sederhana karena hanya memiliki satu buah metode abstrak utama tanpa argumen, yaitu run(), yang berfungsi sebagai wadah untuk meletakkan blok kode operasional. Pada praktikum ini, interface Runnable dimanfaatkan secara kreatif untuk proses benchmarking. Dengan memanfaatkan Lambda Expression () -> {}, kita dapat membungkus serangkaian operasi array atau ArrayList untuk dikirimkan sebagai sebuah argumen ke dalam fungsi pencatat waktu.

4) Efek JIT Compiler, Fase Warm-Up, dan Scaling Loop

Berdasarkan sumber referensi mengenai arsitektur eksekusi Java, anomali performa pada tahap

awal perjalanan program sangat dipengaruhi oleh mekanisme Just-In-Time (JIT) Compiler di dalam Java Virtual Machine. Azul Systems dalam kajian literatur teknisnya mengenai masalah Scaling Loop menjelaskan bahwa saat sebuah aplikasi Java pertama kali dieksekusi, JIT Compiler akan memakan daya komputasi (CPU) yang masif untuk memprofilkan dan menerjemahkan bytecode menjadi bahasa mesin tingkat rendah (machine code) secara dinamis. Fase awal yang padat komputasi ini dikenal sebagai periode warm-up. Dalam skala industri, beban kompilasi awal ini kerap memicu lonjakan utilisasi CPU secara sementara yang dapat mengelabui sistem infrastruktur otomatis (auto-scaling). Secara fundamental, metrik waktu pada perjalanan instruksi pertama dalam lingkungan JVM tidak mempresentasikan kecepatan komputasional algoritma secara murni, melainkan merupakan akumulasi dari latensi sistematis akibat proses penerjemahan kode, inisialisasi ruang memori, dan pemuatan kelas .

Jenis Operasi	Waktu
Waktu Operasi Array	25 ms
Waktu Operasi Array dengan 10000 data	1 ms
Waktu Operasi ArrayList	7 ms
Waktu Operasi ArrayList dengan 10000 data	11 ms

Fenomena warm-up JIT Compiler ini memberikan implikasi komprehensif terhadap validitas hasil benchmarking komparasi antara primitive array dan ArrayList yang diamati pada praktikum. Pengukuran metrik waktu untuk operasi array biasa diletakkan pada urutan eksekusi pertama, sehingga struktur data tersebut menyerap keseluruhan penalti latensi dari proses warm-up JVM, yang mengakibatkan pencatatan waktu yang jauh lebih lama 25 ms. Sebaliknya, ketika algoritma ArrayList dieksekusi pada blok pengujian selanjutnya, JVM telah berada dalam kondisi teroptimasi secara penuh, di mana jalur eksekusi dan memori telah siap terkompilasi, sehingga waktu yang tercatat murni 7 ms. Oleh karena itu, ketidakakuratan data ini membuktikan bahwa pengukuran performa dalam bahasa berbasis JVM wajib mengimplementasikan iterasi pemanasan pendahuluan sebelum fungsi pencatatan waktu diaktifkan agar mereduksi bias kompilasi sistem awal.

B. Hasil Praktikum

1) Operasi Pada Array

a) Array

Pada Operasi Array ini, saya membuat class ArrayOperations yang mana memiliki method - method static agar reusable di panggilan class manapun.

- Traversal

Traversal pada array java cukup memanfaatkan looping berdasarkan length array untuk mencetak setiap elemen.

```
static void TraverseArray(int[] arr) {
    System.out.println("Traversal Array:");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}
```

- Insertion

Untuk Operasi Insertion saya perlu membuat array baru dengan penambahan lengthnya, karena array primitif lengthnya bersifat statis atau tidak bisa diubah

```
static void Insertion(int[] arr, int element) {
    int[] newArr = Arrays.copyOf(arr, arr.length + 1);
    newArr[arr.length] = element;
    TraverseArray(newArr);
}
```

- Deletion

Untuk Operasi Deletion saya perlu membuat array baru dengan pengurangan lengthnya, karena array primitif lengthnya bersifat statis atau tidak bisa diubah. Elemen-elemen dari array lama kemudian disalin ke array baru tersebut, dengan melewati elemen pada posisi yang ingin dihapus

```
static void Deletion(int[] arr, int position) {
    if (position < 0 || position >= arr.length) {
        System.out.println("Invalid posisi");
        return;
    }

    int[] newArr = new int[arr.length - 1];

    System.arraycopy(arr, 0, newArr, 0, position);

    System.arraycopy(arr, position + 1, newArr, position, arr.length - position - 1);

    System.out.println("Array setelah delete element di posisi " + position + ":");
    TraverseArray(newArr);
}
```

- LinearSearch

Pad Operasi Linear Search, langkahnya adalah menelusuri elemen di dalam array satu per satu dari awal hingga akhir menggunakan looping. Setiap elemen akan dicek apakah nilainya sama dengan target yang dicari. Jika nilainya cocok, posisi indeksnya akan langsung ditampilkan dan proses perulangan dihentikan menggunakan break. Sebuah penanda (variabel boolean found) digunakan untuk memastikan jika sampai seluruh array selesai dicek dan elemen tetap tidak ditemukan, program akan memberikan informasi bahwa elemen tersebut tidak ada.

```
static void LinearSearch(int[] arr, int target) {
    System.out.println("Linear Search " + target + ":");
    boolean found = false;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            System.out.println("Element ada di index: " + i);
            found = true;
        }
    }
}
```

```

        break;
    }
}
if (!found) {
    System.out.println("Element tidak ditemukan di array");
}
}

```

- BinarySearch

Pada Operasi Binary Search, logikanya berfokus pada membelah area pencarian menjadi dua bagian secara berulang pada array yang wajib sudah terurut. Pencarian menggunakan dua penunjuk, yaitu batas awal (left) dan akhir (right), untuk mengecek nilai tengah (mid) pada setiap perulangannya. Jika elemen pada indeks tengah sama dengan target, posisi indeks langsung ditampilkan dan perulangan dihentikan dengan break. Namun, jika elemen tengah lebih kecil dari target, area pencarian digeser ke setengah bagian kanan ($\text{left} = \text{mid} + 1$), sedangkan jika lebih besar, area pencarian digeser ke setengah bagian kiri ($\text{right} = \text{mid} - 1$). Proses ini terus berjalan selama area pencarian valid ($\text{left} \leq \text{right}$), dan jika hingga akhir elemen tetap tidak ditemukan, penanda boolean found akan memicu informasi bahwa elemen tidak ada di dalam array.

```

static void BinarySearch(int[] arr, int target) {
    System.out.println("Binary Search " + target + ":");
    int left = 0;
    int right = arr.length - 1;
    boolean found = false;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            System.out.println("Element terletak di index: " + mid);
            found = true;
            break;
        }

        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    if (!found) {
        System.out.println("Element tidak ditemukan di array");
    }
}

```

- Sorting

Logika pengurutan pada kode ini bekerja dengan cara membandingkan dua angka yang bersebelahan secara terus-menerus. Lewat nested looping, program menelusuri isi array lapis demi lapis. Setiap kali menemukan angka sebelah kiri yang lebih besar dari angka di kanannya ($arr[j] > arr[j + 1]$), posisinya akan langsung ditukar menggunakan bantuan variabel penampung sementara

```
static void Sort(int[] arr, boolean traverseAfterSort) {
    System.out.println("Sorting Array");
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    if (traverseAfterSort) {
        TraverseArray(arr);
    }
}
```

b) ArrayList

Pada Operasi ArrayList ini, saya membuat class ArrayListOperations yang mana memiliki method - method static agar reusable di panggilan class manapun.

- Traversal

Logika utama dari kode ini adalah membaca dan menampilkan seluruh data yang ada di dalam ArrayList secara berurutan. Karena struktur data yang digunakan adalah ArrayList dan bukan array biasa, kita menggunakan metode `.size()` untuk mengetahui batas akhir jumlah elemennya. Perulangan kemudian dijalankan mulai dari indeks 0 hingga elemen terakhir.

```
static void TraverseArray(ArrayList<Integer> arr) {
    System.out.println("Traversal ArrayList:");
    for (int i = 0; i < arr.size(); i++) {
        System.out.print(arr.get(i) + " ");
    }
    System.out.println();
}
```

- Insertion

Pada operasi Insertion ini, penggunaan ArrayList sangat memudahkan eksekusi karena

ukurannya yang dinamis. Sebelum data dimasukkan, kode akan melakukan validasi keamanan terlebih dahulu untuk memastikan posisi indeks yang dituju masuk akal yaitu tidak bernilai negatif dan tidak melampaui jumlah elemen yang ada saat ini (`arr.size()`). Jika posisinya valid, kita hanya perlu memanggil fungsi bawaan `.add(position, element)`

```
static void Insertion(ArrayList<Integer> arr, int element, int position) {
    if (position < 0 || position > arr.size()) {
        System.out.println("Invalid position for insertion.");
        return;
    }
    arr.add(position, element);
}
```

- Deletion

Sama seperti sebelumnya, deletion pada struktur ArrayList juga menjadi lebih sederhana, pada kode ini melakukan jika posisinya terbukti valid, program tinggal mengeksekusi perintah bawaan `.remove(position)`. Fungsi tersebut secara otomatis akan membuang elemen pada posisi yang dipilih sekaligus menggeser

```
static void Deletion(ArrayList<Integer> arr, int position) {
    System.out.println("Deletion ArrayList");
    if (position < 0 || position >= arr.size()) {
        System.out.println("Invalid posisi");
        return;
    }
    arr.remove(position);
}
```

- Linear Search

Pada pencarian linier ini, prosesnya jauh lebih sederhana karena kita tidak perlu lagi menggunakan looping karena java sudah menyediakan fungsi bawaan `.indexOf(target)` khusus untuk ArrayList. Di belakang layar, metode ini akan menelusuri elemen dari awal secara berurutan untuk mencari nilai yang cocok. Jika elemen berhasil ditemukan, fungsinya akan mengembalikan posisi indeks pertama di mana elemen itu berada.

```
static void LinearSearch(ArrayList<Integer> arr, int target) {
    System.out.println("Linear Search " + target + ":");
    int index = arr.indexOf(target);
    if (index != -1) {
        System.out.println("Element ditemukan di index: " + index);
    } else {
        System.out.println("Element tidak ditemukan di ArrayList");
    }
}
```

- Binary Search

Logika pencarian biner juga menjadi lebih sederhana berkat adanya pustaka utilitas bawaan Java. Daripada mengatur penunjuk batas awal, tengah, dan akhir secara manual, kode ini langsung memanfaatkan metode `Collections.binarySearch(arr, target)`. Syaratnya list harus sudah terurut terlebih dahulu. Metode ini akan langsung mengeksekusi algoritma pembelahan area pencarian yang sangat cepat di balik layar. Jika target ditemukan, ia akan mengembalikan posisi indeks aslinya (bernilai 0 atau positif). Jika target tidak ada di dalam list, ia akan mengembalikan sebuah nilai negatif. Lewat validasi kondisi ≥ 0 , program bisa langsung menentukan status keberhasilan pencarian tersebut.

```
static void BinarySearch(ArrayList<Integer> arr, int target) {
    System.out.println("Binary Search " + target + ":");
    int index = Collections.binarySearch(arr, target);
    if (index >= 0) {
        System.out.println("Element ditemukan di index: " + index);
    } else {
        System.out.println("Element tidak ditemukan di ArrayList");
    }
}
```

- Sort

Untuk proses sorting, kita bisa memanfaatkan function build in bawaan dari ArrayList

```
static void Sort(ArrayList<Integer> arr, boolean traverseAfterSort) {
    System.out.println("Sorting ArrayList");
    Collections.sort(arr);
    if (traverseAfterSort) {
        TraverseArray(arr);
    }
}
}
```

2) Hasil Benchmark 2 tipe Array

```
package Assignment.LAB.Assignment1;

import java.io.OutputStream;
import java.io.PrintStream;
import java.util.ArrayList;
import java.util.List;

public class Comparison {
    static long measureTime(Runnable task) {
        long start = System.nanoTime();
        task.run();
    }
}
```

```

    long end = System.nanoTime();
    return (end - start) / 1_000_000;
}

static void warmup() {
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(OutputStream.nullOutputStream()));
    for (int i = 0; i < 10000; i++) {
        int[] arr = new int[1000];
        for (int j = 0; j < 1000; j++) {
            arr[j] = j + 1;
        }
        ArrayOperations.LinearSearch(arr, 500);
        ArrayOperations.BinarySearch(arr, 500);
        ArrayOperations.Sort(arr, false);

        ArrayList<Integer> arrList = new ArrayList<>();
        for (int j = 0; j < 1000; j++) {
            arrList.add(j + 1);
        }
        ArrayListOperations.LinearSearch(arrList, 500);
        ArrayListOperations.BinarySearch(arrList, 500);
        ArrayListOperations.Sort(arrList, false);
    }
    System.setOut(originalOut);
    System.out.println("Warmup selesai! Mulai eksekusi benchmark...");
}

public static void main(String[] args) {

    // melakukan warmup untuk mengurangi efek JIT compiler
    warmup();

    int[] arr = { 10, 20, 30, 40, 50 };
    ArrayList<Integer> arrList = new ArrayList<>(List.of(10, 20, 30, 40, 50));

    System.out.println("--- LOG EKSEKUSI OPERASI ---");
    System.out.println("Array:");
    long timeArray = measureTime(() -> {
        ArrayOperations.TraverseArray(arr);
        ArrayOperations.LinearSearch(arr, 30);
        ArrayOperations.BinarySearch(arr, 30);
        ArrayOperations.Insertion(arr, 60);
        ArrayOperations.Deletion(arr, 1);
        ArrayOperations.Sort(arr, true);
    });

    long timeArrayLarge = measureTime(() -> {
        int[] largeList = new int[10000];
        for (int i = 0; i < 10000; i++) {
            largeList[i] = i + 1;
        }
        ArrayOperations.LinearSearch(largeList, 500);
        ArrayOperations.BinarySearch(largeList, 500);
        ArrayOperations.Sort(largeList, false);
    });
}

```



```
});
```

```
System.out.println("\nArray List:");
long timeArrayList = measureTime() -> {
    ArrayListOperations.TraverseArray(arrList);
    ArrayListOperations.LinearSearch(arrList, 30);
    ArrayListOperations.BinarySearch(arrList, 30);
    ArrayListOperations.Insertion(arrList, 60);
    ArrayListOperations.Deletion(arrList, 1);
    ArrayListOperations.Sort(arrList, true);
});
```

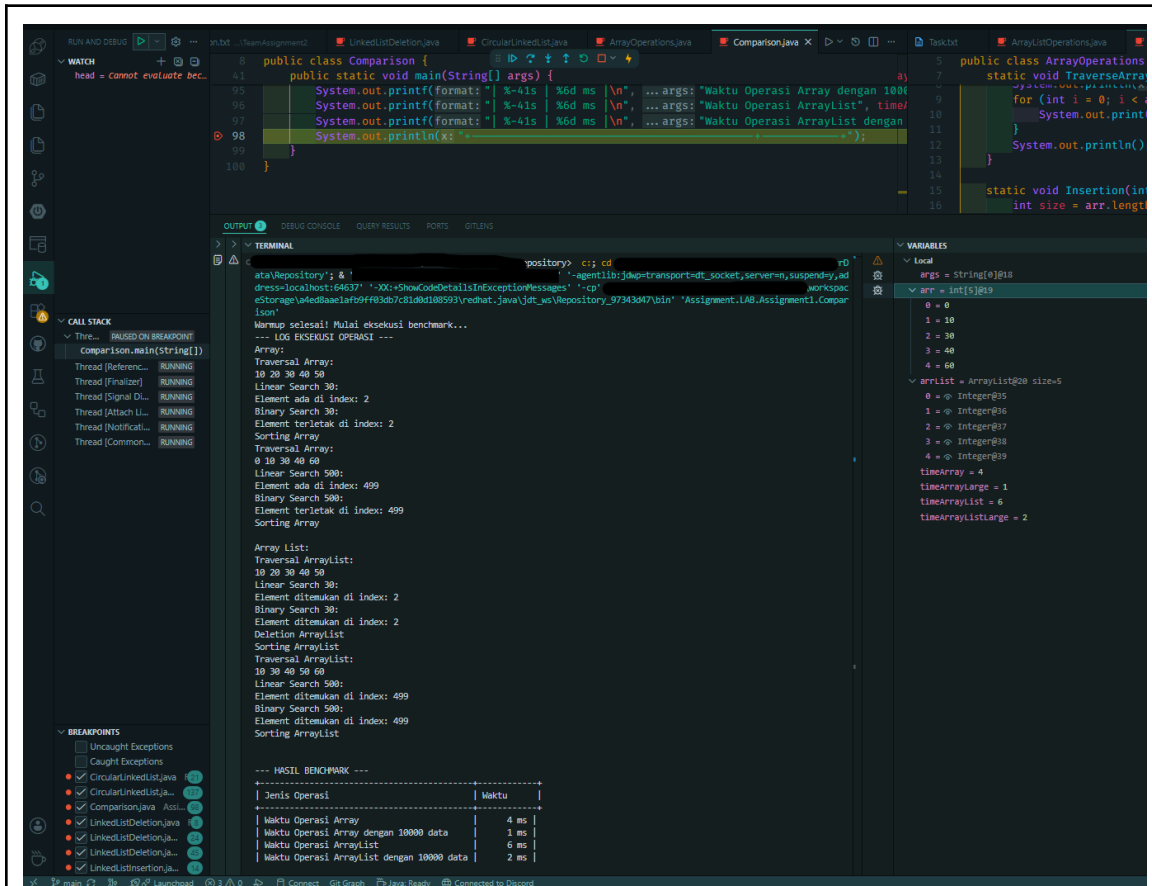
```
long timeArrayListLarge = measureTime() -> {
    ArrayList<Integer> largeArrList = new ArrayList<>();
    for (int i = 0; i < 10000; i++) {
        largeArrList.add(i + 1);
    }
    ArrayListOperations.LinearSearch(largeArrList, 500);
    ArrayListOperations.BinarySearch(largeArrList, 500);
    ArrayListOperations.Sort(largeArrList, false);
});
```

```
System.out.println("\n\n--- HASIL BENCHMARK ---");
System.out.println("+-----+-----+");
System.out.printf("| %-4s | %-10s \n", "Jenis Operasi", "Waktu");
System.out.println("+-----+-----+");
System.out.printf("| %-4s | %-6d ms \n", "Waktu Operasi Array", timeArray);
System.out.printf("| %-4s | %-6d ms \n", "Waktu Operasi Array dengan 10000 data",
timeArrayLarge);
System.out.printf("| %-4s | %-6d ms \n", "Waktu Operasi ArrayList", timeArrayList);
System.out.printf("| %-4s | %-6d ms \n", "Waktu Operasi ArrayList dengan 10000 data",
timeArrayListLarge);
System.out.println("+-----+-----+");
}
}
```

Untuk melakukan benchmark performa antara primitive array dan ArrayList, saya merancang fungsi terpisah bernama `measureTime` yang menerima parameter `Runnable`. Cara ini memastikan proses pengukuran waktu eksekusi kode menjadi lebih rapi dan dapat digunakan kembali untuk berbagai blok operasi tanpa perlu menulis ulang logika perhitungannya. Kemudian, untuk mengantisipasi anomali latensi yang disebabkan oleh proses pemanasan dari Just-In-Time (JIT) Compiler di lingkungan JVM, saya mengimplementasikan fungsi `warmup()`. Di dalam fungsi ini, saya menjalankan simulasi operasi secara berulang 10.000 iterasi pada kedua struktur data sebelum pengukuran waktu sebenarnya dimulai. Tujuannya adalah untuk memaksa JIT Compiler memuat class, melakukan optimalisasi memori, dan menerjemahkan bytecode menjadi bahasa mesin secara utuh di awal eksekusi.

Selama proses warm-up berlangsung, saya juga memanfaatkan `null stream` `OutputStream.nullOutputStream()` untuk mengalihkan semua output konsol agar tidak membebani sistem I/O layar, memastikan proses pemanasan berjalan cepat. Dengan membiarkan mesin Java mencapai titik operasi optimalnya terlebih dahulu, kita dapat menghilangkan bias kompilasi dan memastikan bahwa waktu benchmark yang tercatat pada akhirnya murni merepresentasikan kecepatan algoritma dari masing-masing struktur data tersebut.

Berikut hasil compile dari class Comparison



```

public class Comparison {
    public static void main(String[] args) {
        System.out.printf(format: "%-41s | %-6d ms | \n", ... args: "Waktu Operasi Array dengan 1000 data", time: 4);
        System.out.printf(format: "%-41s | %-6d ms | \n", ... args: "Waktu Operasi ArrayList", time: 6);
        System.out.printf(format: "%-41s | %-6d ms | \n", ... args: "Waktu Operasi ArrayList dengan 10000 data", time: 1);
        System.out.println(" ");
    }
}

```

OUTPUT

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi | Waktu |
+-----+-----+
| Waktu Operasi Array | 4 ms |
| Waktu Operasi Array dengan 10000 data | 1 ms |
| Waktu Operasi ArrayList | 6 ms |
| Waktu Operasi ArrayList dengan 10000 data | 2 ms |
+-----+-----+

```

Pengetesan di bantu dengan tools dari IDE vscode java debugger untuk tracing data dan flow execution code

Hasil Compile 1

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi | Waktu |
+-----+-----+
| Waktu Operasi Array | 4 ms |
| Waktu Operasi Array dengan 10000 data | 1 ms |
| Waktu Operasi ArrayList | 6 ms |
| Waktu Operasi ArrayList dengan 10000 data | 2 ms |
+-----+-----+

```

Hasil Compile 2

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi                | Waktu    |
+-----+-----+
| Waktu Operasi Array          | 3 ms    |
| Waktu Operasi Array dengan 10000 data | 1 ms    |
| Waktu Operasi ArrayList      | 5 ms    |
| Waktu Operasi ArrayList dengan 10000 data | 2 ms    |

```

Hasil Compile 3

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi                | Waktu    |
+-----+-----+
| Waktu Operasi Array          | 3 ms    |
| Waktu Operasi Array dengan 10000 data | 0 ms    |
| Waktu Operasi ArrayList      | 4 ms    |
| Waktu Operasi ArrayList dengan 10000 data | 1 ms    |

```

Hasil Compile 4

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi                | Waktu    |
+-----+-----+
| Waktu Operasi Array          | 3 ms    |
| Waktu Operasi Array dengan 10000 data | 0 ms    |
| Waktu Operasi ArrayList      | 4 ms    |
| Waktu Operasi ArrayList dengan 10000 data | 2 ms    |

```

Hasil Compile 5

```

--- HASIL BENCHMARK ---
+-----+-----+
| Jenis Operasi                | Waktu    |
+-----+-----+
| Waktu Operasi Array          | 3 ms    |
| Waktu Operasi Array dengan 10000 data | 0 ms    |
| Waktu Operasi ArrayList      | 4 ms    |
| Waktu Operasi ArrayList dengan 10000 data | 1 ms    |

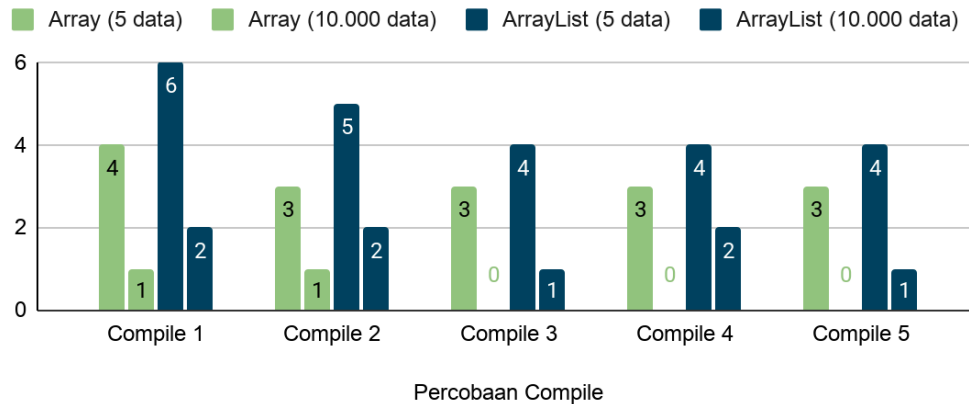
```

3) Kesimpulan

Pada praktikum ini, dapat disimpulkan bahwa penggunaan utility atau fungsi built-in bawaan Java (seperti ArrayList) memang sangat mempermudah programmer dalam mengelola struktur data

yang dinamis dan mempercepat siklus penulisan kode. Namun, kepraktisan ini tetap membawa trade-off terhadap efisiensi performa komputasi.

Hasil Benchmarking Array vs ArrayList



Berdasarkan lima kali percobaan compile dengan metode warm-up yang benar, operasi menggunakan array primitif dengan 10.000 data terbukti lebih konsisten dan cepat dimana memakan waktu antara 0 hingga 1 ms, dibandingkan dengan operasi identik pada ArrayList yang sedikit lebih lambat berada di kisaran 1 hingga 2 ms. Walaupun selisih waktu tersebut kini terlihat kecil, hal ini tetap mengonfirmasi keberadaan overhead struktural pada ArrayList akibat proses auto-boxing dan manajemen memori berbasis objek. Selain itu, fluktuasi minor pada rentang waktu antar percobaan juga membuktikan bahwa hasil benchmark tidak bersifat absolut, melainkan sangat dipengaruhi oleh faktor eksternal. Variabel seperti kapabilitas hardware perangkat keras, beban tugas background processes pada sistem operasi, serta dinamisnya alokasi sumber daya JVM pada saat runtime turut menentukan hasil akhir eksekusi. Oleh karena itu, dalam rekayasa sistem yang menangani volume data masif secara kritis atau membutuhkan latensi eksekusi seminimal mungkin, struktur array primitif tetap menjadi fondasi arsitektur komputasi yang paling tangguh dan presisi.

AI Usage Declaration:

<https://drive.google.com/file/d/1U92Jw9BWPR1p8Xp1ErkAt4Aj6JjkBDg/view?usp=sharing>

Full Source Code:

<https://github.com/Shefiree/Data-Structures-and-Algorithm-Analysis-COSC6025036/tree/main/Assignment/LAB/Assignment1>

Sumber Referensi

GeeksforGeeks. (n.d.). Java.util Package in Java. Diakses pada 4 Juli 2026,
<https://www.geeksforgeeks.org/java/java-util-package-java/>

Oracle. (n.d.). Primitive Data Types (The Java™ Tutorials). Diakses pada 4 Juli 2026,
<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Oracle. (n.d.). Runnable (Java Platform SE 8). Diakses pada 4 Juli 2026,
<https://docs.oracle.com/javase/8/docs/api/java/lang/Runnable.html>

Azul Systems. (n.d.). *Java warmup and the scaling loop problem*. Azul Blog. Diakses pada 5 Juli 2026, <https://www.azul.com/blog/java-warmup-and-the-scaling-loop-problem/>